error, which is fine, but who knows what damage further changes might have made? Better to fail early and either fix the code or change the design. (Hopefully I'm making a case for the use of type declarations here!)

Two things to note with the `void` return type. First, a `void` function will *always* return a value (and type) of `null`; be aware of this when using `void` returning functions in expressions. Second, `void` can never be a valid return type for functions that return by reference, even if the value of the reference is `null`.

References and Return Types

Functions that handle variable references (see earlier sections of this chapter) have some caveats when it comes to type declaration. First, let's look at pass-by-reference:

```
function breakMyArray(array &$things)
{
    $things = 'Oops!';
}

$arr = ['some', 'lovely', 'things'];
breakMyArray($arr);
print "Attempting second call to breakMyArray()" . PHP_EOL;
breakMyArray($arr);

Output:
Attempting second call to breakMyArray()
Uncaught TypeError: breakMyArray(): Argument #1 ($things) must be of type
array, string given
```

Oh dear. It should be apparent why this happened, after all we changed our `array` into a `string` with the first call to `breakMyArray()` so of course it's not a valid type on the second. What's perhaps less obvious is that the type isn't enforced by the declaration of `array` on the `&$things` argument. The type checking is only done on function entry, not exit. Perhaps we can enforce it by specifying a return type for the function:

```
function breakMyArray(array &$things): array
{
    $things = 'Oops!';
}
```